CIVITA · Entrega de Ingeniería de Software II

Versión 1.1.742. Android nativo con Kotlin, Jetpack Compose y Firebase. Autoría de interfaz: By: Joan Erazo v.10742.

Problema y comportamiento

CIVITA permite compartir novedades ciudadanas de Popayán, consultar categorías, publicar reportes y participar mediante comentarios, validaciones y apoyos. El acceso se realiza con Google o correo/contraseña mediante Firebase Authentication. Los datos documentales se almacenan en Cloud Firestore; los archivos de evidencia usan Cloud Storage cuando el bucket está habilitado.

Arquitectura implementada

Compose representa los estados observables. AuthViewModel coordina credenciales y sesión; CivitaViewModel carga catálogo/configuración y publica reportes; CommunityViewModel gestiona perfil, preferencias, detalle, participación y buzón. ReportRepository es el contrato de reportes y FirebaseReportRepository su adaptador. CommunityActions es el contrato de acciones de las pantallas de detalle y preferencias; CommunityViewModel proporciona la implementación Firebase. CommunityRepository accede a Firestore y Storage. AppContainer inicializa los SDK con el google-services.json actualizado.

La app separa estado de presentación y acceso a datos. Los listeners se retiran al cancelar sus flujos y cerrar sesión. Las operaciones muestran progreso y errores; el estado busy impide envíos simultáneos. Los plazos de espera no equivalen a un rechazo del servidor: un envío puede haberse confirmado después de perderse la respuesta.

Correspondencia con el MER original

Se preservan ambos archivos draw.io originales en docs/database. La adaptación final añade apoyos, suscripciones y capas del MER relacional que no figuraban completos en el esquema documental anterior. UID sustituye al identificador SQL del usuario; las relaciones se expresan con IDs o rutas, no con joins. Los puntos usan GeoPoint, las coordenadas de reportes se acompañan de geohash estándar y los campos JSON de capas se guardan como String JSON.

Las rutas completas, atributos, tipos, validaciones y permisos están en firebase/SCHEMA.md y schema.json. Un documento de configuración no equivale a una colección de actividad: Firestore materializa cada colección cuando se escribe su primer documento. No se añaden usuarios, reportes o eventos ficticios a producción para aparentar una base poblada.

Integridad y seguridad

Un reporte nuevo se crea con su historial inicial en una transacción. El ID de publicación se conserva al reintentar el mismo borrador. La categoría debe estar activa, el autor debe coincidir con el UID autenticado y los timestamps de alta provienen del servidor. Las coordenadas son opcionales; nunca se inventa una posición GPS.

Cada validación y apoyo usa el UID como ID dentro del reporte: evita votos y apoyos duplicados. Se prohíbe autovalidar y el peso del voto es fijo. El comentario toma el nombre público del perfil y conserva un ID estable en reintentos. El ocultamiento de un comentario es una operación de presentación para moderadores; su contenido permanece en la base para auditoría.

El cambio de estado exige un custom claim role=moderator/admin. Actualiza reporte, historial, acción de moderación y notificación al autor en un único commit. Editar users.role no concede privilegios. El perfil y el correo son privados; el buzón y las preferencias sólo son accesibles para su propietario.

Las evidencias se cargan en Cloud Storage y después se confirma su metadata con un contador transaccional de máximo tres archivos. MIME permitido: JPEG/PNG/MP4; tamaño máximo: 10 MiB. Si falla el commit, se intenta limpiar el archivo sin metadata. El protocolo de dos servicios no es una transacción distribuida: un cierre abrupto puede dejar un archivo huérfano; antes de un despliegue de gran escala debe agregarse limpieza administrativa periódica. El acceso a Storage requiere bucket y reglas operativos.

Funcionalidades para la sustentación

Primera demostración: registrar una cuenta propia por correo, cerrar sesión, iniciar sesión nuevamente y verificar perfil/preferencias persistentes. Mostrar validación de correo y contraseña. Probar Google también en un teléfono con Play Services y SHA registrada.

Segunda demostración: publicar un reporte real con categoría, título, descripción, referencia y afectación. Verlo en Comunidad, abrir el detalle y añadir comentario. Con otra cuenta, confirmar/desmentir y apoyar. Mostrar en Firestore el documento y sus subcolecciones. Cerrar y abrir la app para demostrar persistencia.

Demostración adicional: un moderador autorizado cambia el estado indicando motivo. Mostrar historial, moderationActions y notificación privada del autor. No asignar el rol desde el cliente ni presentar una simulación como acceso real.

Diagramas y documentación

Las vistas UML incluyen dominio, participación, cuentas, repositorios, coordinación, estados y autenticación. El diagrama de componentes muestra las responsabilidades y las dependencias entre app, SDK y servicios. El MER se divide en reportes, cuenta/territorio y autorías para conservar cardinalidades legibles. Dos vistas detallan el modelo documental. Las secuencias explican publicación y moderación.

Notación: UML 2.5.1 para clases/componentes/secuencias; pata de cuervo para MER. + y - representan visibilidad; tipos nullable conservan ?. Las líneas discontinuas con triángulo representan realización de interfaces, las continuas con triángulo representan herencia y las flechas discontinuas representan dependencias. No se dibujan rombos por conveniencia estética: las referencias compartidas de Kotlin y las subcolecciones Firestore no demuestran por sí mismas composición con borrado en cascada.

Límites que deben explicarse

El mapa principal es ilustrativo, sin GPS ni navegación. Las suscripciones por zona guardan preferencias, pero no hay un servicio de envío push desplegado. deviceTokens dispone de esquema/reglas sin registro automático. Los recuentos de validación se muestran en el detalle; confidence y trustLevel permanecen reservados, sin cálculo automático afirmado. Los eventos y puntos oficiales se crean con la herramienta administrativa, nunca como incidentes ficticios de prueba.

La base ya fue verificada administrativamente según la salida de Cloud Shell compartida por el usuario. Las reglas ampliadas de esta versión requieren nueva publicación. La compilación y los tests locales no sustituyen verificar Authentication y Firestore desde un dispositivo real. Consultar VALIDACION.md para evidencia exacta y bloqueos.

Referencias de notación y plataforma

UML 2.5.1 (OMG): https://www.omg.org/spec/UML/2.5.1 . Modelo documental Firestore: https://firebase.google.com/docs/firestore/data-model . Transacciones: https://firebase.google.com/docs/firestore/manage-data/transactions . Arquitectura Android: https://developer.android.com/topic/architecture . Los diagramas reflejan la implementación de esta entrega; estas referencias definen la notación y las responsabilidades generales, no certifican el funcionamiento del proyecto.